La Plateforme
La grande école du numérique pour tous

KickOff
Lancement
Correspondances





Correspondances

(1 semaine)

Refonte du système de billetterie ferroviaire régional :

- Modélisation avancée
- Migration de schéma en production et passage à l'échelle PostgreSQL 16 sous forte volumétrie.

Contexte & Objectifs

Un réseau régional modeste mais exigeant

- une trentaine de gares interconnectées, une douzaine de lignes et des circulations variées (quotidiennes, semaine, week-end, dates exceptionnelles).

- La billetterie doit absorber sans latence plusieurs centaines de milliers de réservations annuelles avec un haut niveau de cohérence.

Niveau professionnel exigé

Au-delà du simple CRUD :

le livrable central n'est pas l'interface utilisateur, mais l'architecture de données, sa robustesse contractuelle et sa résilience sous charge.

Le code technique (tables, colonnes, contraintes, entités JPA) est intégralement rédigé en anglais selon les standards de l'industrie.



Les trois piliers techniques

Modélisation Formelle

- Conception rigoureuse du domaine ferroviaire à travers le cycle complet **MCD → MLD → MPD**, validé et commité avant la moindre ligne de code applicatif.

Migration en Production

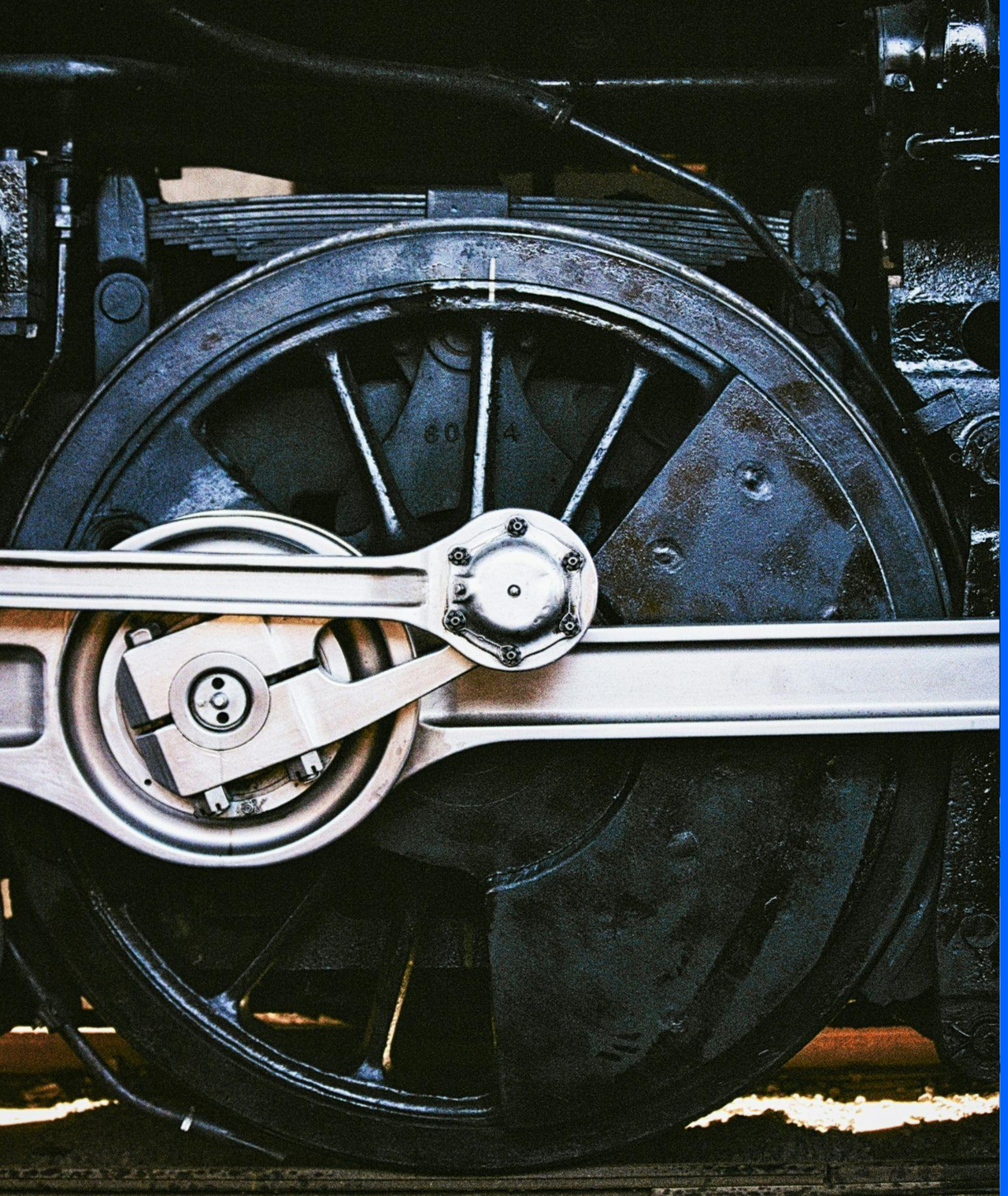
- Évolution continue du schéma avec **Flyway ou Liquibase** sans jamais détruire les données existantes. Nettoyage ciblé et gestion des ruptures d'intégrité réelles.

Performance sous Charge

- Optimisation factuelle sur **500 000+ lignes** : plans d'exécution EXPLAIN ANALYZE, indexation composée et éradication du piège N+1 JPA.



EXPLAIN ANALYZE est un outil de diagnostic des performances SQL



Modélisation du domaine métier

Complexité ferroviaire & points d'attention :

- **Desserte porteuse de données :**

  Association riche entre train et gare, intégrant les heures exactes d'arrivée et départ (trains directs vs omnibus).

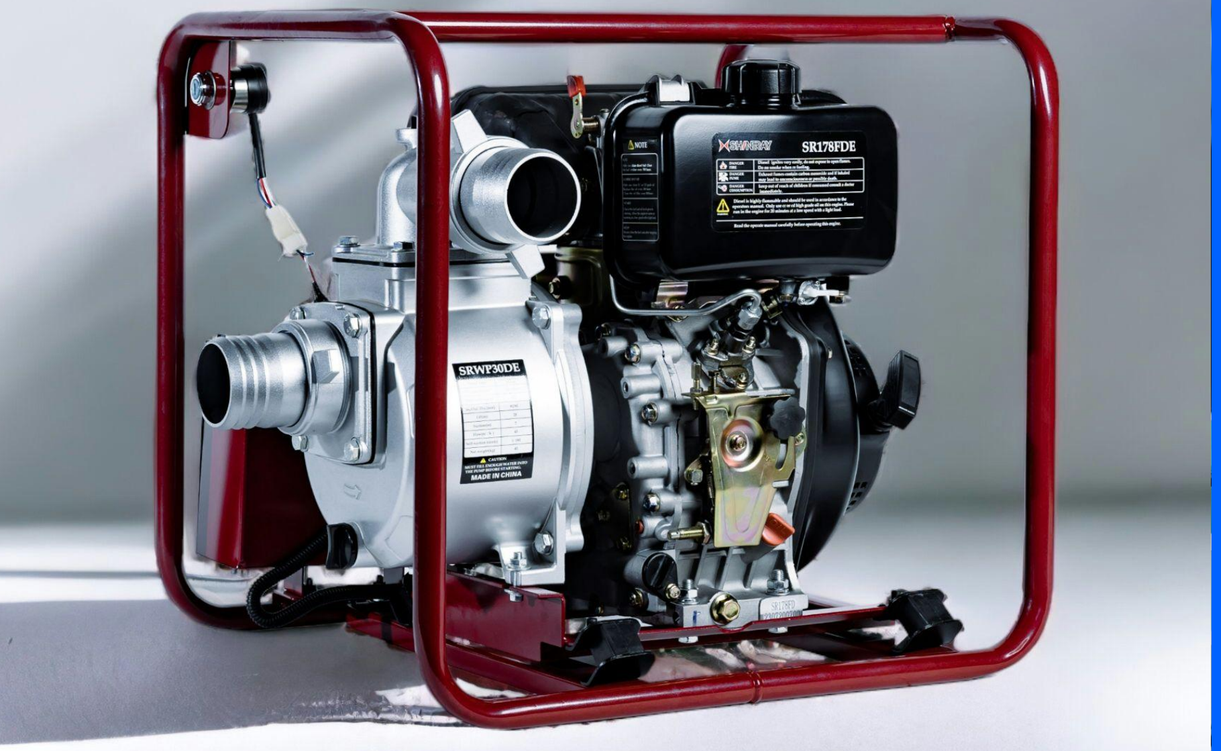
- **Tarification multi-dimensionnelle :**

  Dépend de la distance, de la classe (1ère/2nde), de la tranche horaire et du profil voyageur.

- **Règle clé des réservations :**

  Une place est réservée sur un *segment de trajet* précis. Lorsqu'un passager descend à mi-parcours, la place redevient réservable immédiatement.

generate_series() est une fonction de **PostgreSQL** qui permet de générer **automatiquement** une série de valeurs (nombres, dates ou heures).



Générateur

Génération d’au moins 500 000 réservations :

- **Le générateur s'écrit en Java ou en SQL :**
  - Le SQL est parfaitement adapté ici, et **generate_series** fait partie de ce que vous devez savoir manipuler
  - réparties sur une année de circulation, avec des jours **plus chargés que d'autres**
  - des **données cohérentes** : une réservation pointe vers une circulation qui existe réellement ce jour-là.
  - Le **générateur** est un **livrable**. Il doit se relancer proprement, documenté dans le **README**



“La région veut pouvoir consulter l'historique des prix. Aujourd'hui, quand un tarif change, l'ancien disparaît. Nous voulons désormais conserver tous les tarifs passés, avec leur période de validité, et pouvoir répondre à la question : combien coûtait ce trajet le 3 mars dernier ? Bien entendu, les réservations existantes doivent rester intactes et continuer à pointer vers le prix qui leur a été appliqué au moment de l'achat. “

Evolution du CDC

Ce que cela implique :

- une migration versionnée avec **Flyway** ou **Liquibase** ;
- une migration qui transforme les données existantes, pas seulement la structure
- une base qui se migre et ne se recrée pas. Supprimer et régénérer est un échec sur cette étape, même si le résultat final est correct.

Performances

1. Performance & EXPLAIN ANALYZE

- **Seq Scan vs Index Scan :** Passage d'un parcours complet de la table (*lent,* O(N)) à une recherche ciblée dans un B-Tree (*rapide,* O(log N)).
- **Cost vs Time :** Comparaison entre l'estimation théorique du planificateur (cost) et le temps réel mesuré (actual time).
- **Gain mesuré :** Réduction du temps d'exécution (ex: passage de 120 ms à 2 ms).

2. Stratégie d'Indexation

- **Création d'index :** Justifiée par des requêtes fréquentes sur des clauses WHERE, JOIN ou ORDER BY.
- **Index composés :**
  - Ordre des colonnes basé sur la **sélectivité** (la colonne la plus discriminante en premier).
  - Respect de la **règle du préfixe** (filtrage sur A ou A+B).

3. Résolution du Problème N+1 (JPA)

- **Diagnostic :** Identification via les logs SQL (ex: 1 requête principale + N requêtes pour charger les relations).
- **Correction appliquée :** Utilisation de JOIN FETCH (ou @EntityGraph) dans le repository.
- **Preuve :** Réduction du volume de requêtes à **1 seule requête SQL** avec jointure.

4. Requêtes SQL Avancées

- **5 requêtes complexes :** Mêlant jointures multiples, agrégations (GROUP BY / HAVING).
- **Fonction Fenêtre (WINDOW FUNCTION) :** Usage de DENSE_RANK() OVER (PARTITION BY ... ORDER BY ...) pour établir un classement des réservations les plus populaires.

Le dossier de mesure est un livrable à part entière.

Performances
(explain)

Respect de la **règle du préfixe** (filtrage sur A ou A+B).

Imagine un répertoire classé d'abord par nom, puis par prénom.

Chercher un nom est facile, chercher un nom et un prénom aussi. Mais chercher uniquement un prénom est plus compliqué !



Problème N+1



- 1 requête SQL pour récupérer une liste d'éléments.
- Puis N requêtes supplémentaires pour récupérer les détails de chaque élément.

Exemple : récupérer 100 clients = 1 requête pour les clients + 100 requêtes pour leurs commandes = **101 requêtes au lieu d'une seule ou de quelques-unes**.

@EntityGraph

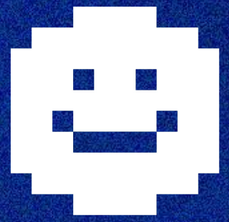
Annotation JPA qui permet de **charger à l'avance les relations d'une entité** (ex. : client et commandes) pour éviter le problème N+1, sans écrire explicitement un JOIN FETCH

Window Function

Ajoute une information (Colonne temporaire) calculée à chaque ligne, sans supprimer les lignes existantes.

DENSE_RANK()

Est une fonction SQL qui attribue un classement aux lignes, sans sauter de rang en cas d'égalité.



La Plateforme